

- Why This Project Matters — And Why It's Great for You
  - The Industry Opportunity
  - What You'll Actually Do
  - Why This Beats a Software Internship
  - Career Paths This Enables
- Fine-Tuning Small Language Models for High-Performance GPU Code Generation
- Project Summary
- Research Objectives
- Student Responsibilities
- Anticipated Outcomes
- Required Qualifications
- Preferred / Asset Qualifications
- Supervision & Training Environment
- 📅 Milestones and Deliverables
- 🎓 Learning Outcomes
- 🔗 References

## Why This Project Matters — And Why It's Great for You

### The Industry Opportunity

LLMs can write code — but can they write *fast* code? Today's models generate functionally correct programs but ignore the memory hierarchies, thread scheduling, and hardware constraints that determine real-world performance. **This is the next frontier in AI for code.**

Companies like **NVIDIA, Google, Meta, and OpenAI** are racing to build AI systems that understand hardware. GPU programming remains notoriously difficult — even expert engineers spend weeks tuning kernels. A model that can generate optimized CUDA code would transform how software is written.

### What You'll Actually Do

You'll train language models to think like GPU architects. Starting with a 3B–7B parameter model, you'll:

- Build a dataset of optimized CUDA kernels with performance annotations
- Fine-tune using LoRA/QLoRA on an **RTX 6000 Blackwell (96GB VRAM)**
- Implement reinforcement learning with speedup as the reward signal
- Profile generated code using NVIDIA Nsight to understand what the model learns

This isn't "prompt engineering" — you're teaching models to reason about memory coalescing, shared memory tiling, and tensor cores.

### Why This Beats a Software Internship

| Typical SWE Internship | This Research Project                   |
|------------------------|-----------------------------------------|
| Use existing ML tools  | <b>Train models on novel objectives</b> |
| Write application code | Write code that writes code             |

| Typical SWE Internship    | This Research Project                                    |
|---------------------------|----------------------------------------------------------|
| Learn one framework       | GPU architecture + LLMs + RL                             |
| Limited research exposure | <b>Publication potential + strong grad school letter</b> |

Career Paths This Enables

This project is ideal for students targeting:

- **AI research** (PhD programs in ML, program synthesis, or systems)
- **ML infrastructure teams** (PyTorch, JAX, Triton compiler teams)
- **GPU/AI accelerator roles** (NVIDIA, AMD, Intel, startups)
- **AI labs** (OpenAI, Anthropic, DeepMind, Cognition AI)

Fine-Tuning Small Language Models for High-Performance GPU Code Generation

**Supervisor:** Dr. Arrvindh Shriraman **Department:** School of Computing Science **Institution:** Simon Fraser University **Position Type:** NSERC Undergraduate Student Research Award (USRA) **Duration:** 16 Weeks (Full-Time) **Location:** SFU Burnaby Campus **Start Date:** Summer 2026 (Tentative)

Project Summary

Large Language Models (LLMs) have demonstrated strong capability in generating functionally correct source code. However, most existing work evaluates correctness rather than **performance**. In modern high-performance computing, particularly on GPUs, performance depends critically on memory hierarchy utilization, thread scheduling, shared memory tiling, warp-level communication, and asynchronous data transfer.

This project investigates whether **compact language models (3B–7B parameters)** can be trained to generate **high-performance CUDA kernels**, not merely correct ones. Inspired by recent work such as Cognition AI’s Kevin-32B model, this project will explore reinforcement learning, multi-turn refinement strategies, and performance-based reward shaping, with the goal of teaching models to reason about GPU hardware and produce optimized code.

The student will develop an end-to-end pipeline including dataset construction, fine-tuning, performance evaluation, and reinforcement learning for iterative code refinement. Training and experimentation will run on an NVIDIA RTX 6000 Blackwell workstation (96GB VRAM).

Research Objectives

- Develop a curated dataset of GPU kernel optimizations with profiling and performance annotations
- Fine-tune small code LLMs using LoRA/QLoRA
- Implement multi-turn reinforcement learning with performance-based reward signals
- Evaluate generated kernels against naive implementations, auto-generated baselines, and expert-optimized code

- Analyze optimization strategies learned by the model (e.g., coalescing, tiling, async copy, tensor core usage)
- 

## Student Responsibilities

The USRA student will:

- Set up the experimental and benchmarking environment
- Develop preprocessing and training pipelines
- Implement correctness and performance evaluation harnesses
- Run fine-tuning and reinforcement learning experiments
- Profile kernels using NVIDIA Nsight and interpret performance results
- Prepare intermediate presentations and a final research report

Regular supervision meetings and mentorship will be provided.

---

## Anticipated Outcomes

By the end of the project, the student will:

- Gain hands-on experience in CUDA programming and GPU architecture
  - Learn modern LLM fine-tuning techniques (LoRA/QLoRA)
  - Gain experience with reinforcement learning for program synthesis
  - Develop strong empirical research and experimental methodology skills
  - Contribute to potential research publications and technical artifacts
- 

## Required Qualifications

- Senior undergraduate standing in Computing Science, Engineering, or related field
  - Strong programming skills in Python
  - Coursework in computer systems or architecture (e.g., CMPT 295 or equivalent)
  - Familiarity with basic machine learning concepts
- 

## Preferred / Asset Qualifications

(Not required, but beneficial)

- Experience with CUDA or GPU programming
  - Experience with PyTorch
  - Familiarity with reinforcement learning
  - Prior experience with performance profiling or benchmarking
- 

## Supervision & Training Environment

The student will work within a supportive research environment focused on computer architecture and ML systems. The project is designed to be challenging yet achievable for motivated senior undergraduates.

Structured mentorship, onboarding, and weekly meetings will help ensure successful progress.

 Milestones and Deliverables

| Milestone                            | Timeline    | Deliverables                                                                                                                                    | Success Criteria                                                    |
|--------------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| M1: Environment & Baseline Setup     | Weeks 1-2   | Blackwell environment configured; KernelBench evaluation harness running; baseline Qwen/Kimi model benchmarked                                  | Successfully run 50+ kernel generation tasks with profiling metrics |
| M2: Dataset Curation & Preprocessing | Weeks 3-5   | Curated dataset of 200+ optimization examples with before/after code pairs; memory profiling annotations; difficulty-stratified training splits | Dataset validated with cache metrics and speedup labels             |
| M3: Single-Turn Fine-Tuning Pipeline | Weeks 6-8   | LoRA/QLoRA fine-tuning pipeline for 3B-7B models; initial trained checkpoint; correctness and speedup evaluation                                | Model achieves >60% correctness on held-out tasks                   |
| M4: Multi-Turn RL Training           | Weeks 9-12  | GRPO-based multi-turn training loop with iterative refinement; performance-based reward shaping; training stability analysis                    | Model demonstrates iterative improvement over 4+ refinement steps   |
| M5: Optimization Technique Coverage  | Weeks 13-14 | Evaluation across memory coalescing, shared memory, constant memory, tensor cores, async copy patterns; ablation studies                        | Generated code covers all major optimization categories             |
| M6: Final Evaluation & Documentation | Weeks 15-16 | Comprehensive benchmark report; trained model weights; technical documentation; demo notebook                                                   | Average speedup >2x over naive implementations on test set          |

 Learning Outcomes

Through this project, you will develop expertise in:

- **CUDA Programming & GPU Architecture:** Thread hierarchies, memory coalescing, shared memory tiling, warp-level intrinsics (`__shfl_sync`, `__syncwarp`), and occupancy optimization
- **Tensor Core Programming:** WMMA API for matrix operations, mixed-precision computation, and tensor core scheduling strategies
- **Asynchronous Memory Hierarchy:** `cp.async` pipelines, double-buffering techniques, and latency hiding through overlapped compute/memory operations
- **LLM Fine-Tuning Techniques:** LoRA/QLoRA parameter-efficient training, gradient checkpointing, and memory-efficient training on large models
- **Reinforcement Learning for Code:** GRPO (Group Relative Policy Optimization), multi-turn reward shaping, and preventing reward hacking in code generation

- **Performance Profiling:** Nsight Compute metrics interpretation, roofline analysis, and bottleneck identification
- 

## References

- [Kevin-32B: Multi-Turn RL for Writing CUDA Kernels](#) - Cognition AI
- [KernelBench: Benchmark for LLM Kernel Generation](#) - Stanford Scaling Intelligence
- [Qwen2.5-Coder Technical Report](#) - Alibaba
- [SFU-ARCH Helper](#)
- [SFU-ARCH LLM Resources](#)